

Lecture 6: Generative Models
Variational Autoencoders, GANs, Diffusion
Digits, Blood Cells and Spin Configurations

Deep Learning for Scientific Discovery

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



Recap: Understanding Data without Labels

- PCA and t-SNE: a few coordinates that describe each sample
- k-means and DBSCAN: groups of similar samples
- autoencoders: a learned compression into a latent space; the reconstruction error flags unusual samples
- anomaly detection: unusual blood cells and credit-card fraud flagged by a large reconstruction error

Today: learn the distribution of the data itself, and draw new samples from it.

1

Learning a Probability Distribution



JAGIELLONIAN UNIVERSITY
IN KRAKOW

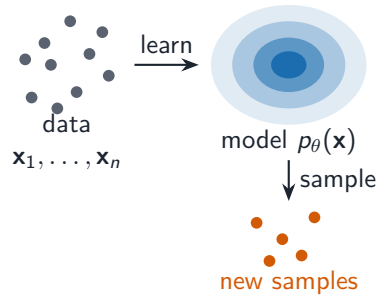
Generating New Data

The data $\mathbf{x}_1, \ldots, \mathbf{x}_n$ are **samples** of an unknown distribution $p(\mathbf{x})$: of handwritten digits, blood-cell images, spin configurations at temperature T.

A **generative model**

- learns an approximation $p_\theta(\mathbf{x})$ of $p(\mathbf{x})$ from the samples (θ: its parameters)
- draws **new samples**: instances not in the data set, but that could have been

In science: new molecules and materials, protein structures, weather ensembles, fast surrogates of simulations.



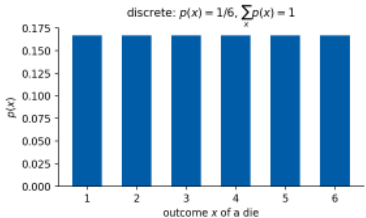
Discrete and Continuous Distributions

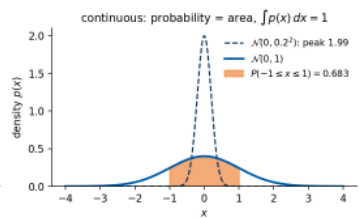
Discrete x (a die, a class, a spin): $p(x)$ is a probability,

$$0 \le p(x) \le 1, \qquad \sum_x p(x) = 1.$$

Continuous x (an intensity, a coordinate): $p(x)$ is a *density*; only areas are probabilities,

$$P(a \le x \le b) = \int_a^b p(x)\,dx, \qquad \int p(x)\,dx = 1$$



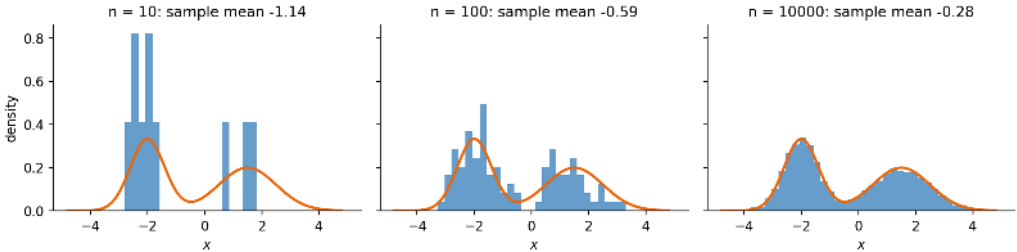


A density can exceed 1 (peak 1.99 for $\sigma = 0.2$). An MNIST image is a point $\mathbf{x} \in \mathbb{R}^{784}$, and $p(\mathbf{x})$ is a density in 784 dimensions.

Expectation and Averages over Samples

The **expectation** of a function f under p, and its estimate from samples $x_1, \ldots, x_n$ of p:

$$\mathbb{E}_{x\sim p}[f(x)] = \int p(x)\, f(x)\, dx \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f(x_i), \qquad \text{std of the estimate} = \frac{\sigma_f}{\sqrt{n}}$$



Orange: $p(x) = \frac{1}{2}\mathcal{N}(x; -2, 0.6^2) + \frac{1}{2}\mathcal{N}(x; 1.5, 1)$, with $\mathbb{E}[x] = -0.25$; blue: histograms of n samples. Every loss in this lecture is an expectation, estimated as an average over a mini-batch (σ_f: standard deviation of $f(x)$ under p).

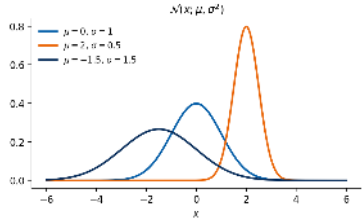
The Gaussian Distribution

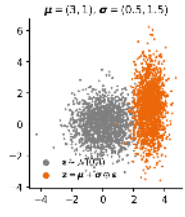
One dimension, mean μ, standard deviation σ:

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$$

d independent coordinates; $\mathcal{N}(0, I)$: all $\mu_k = 0$, $\sigma_k = 1$:

$$\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2) = \prod_{k=1}^{d} \mathcal{N}(z_k; \mu_k, \sigma_k^2)$$



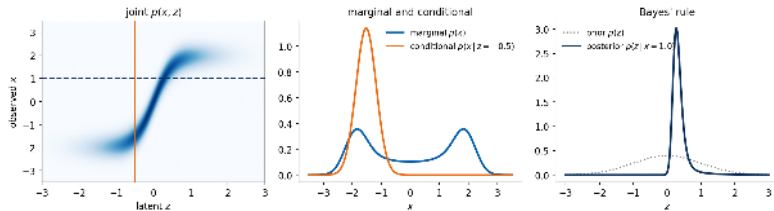


Sampling: draw $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$, then $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ ($\odot$: element-wise product) is a sample of $\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2)$: a shift and a stretch.

Joint, Conditional and Marginal Distributions

An observed x and a hidden (latent) z, with joint distribution $p(x, z)$:

$$\underbrace{p(x) = \int p(x, z)\, dz}_{\text{marginal}} \qquad \underbrace{p(x, z) = p(x \mid z)\, p(z)}_{\text{conditional, product rule}} \qquad \underbrace{p(z \mid x) = \frac{p(x \mid z)\, p(z)}{p(x)}}_{\text{Bayes' rule: posterior}}$$



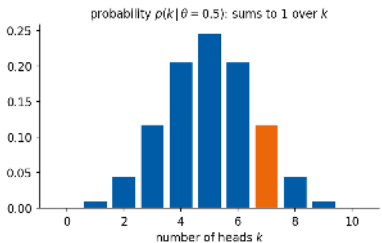
$p(z)$: the *prior*, what we know about z before seeing x; $p(z \mid x)$: the *posterior*, after seeing x. Here $z \sim \mathcal{N}(0, 1)$, $x \mid z \sim \mathcal{N}\big(2 \tanh(2z), 0.35^2\big)$: a two-peaked $p(x)$; observing $x = 1$ narrows the prior (std 1) to a posterior with std 0.25.

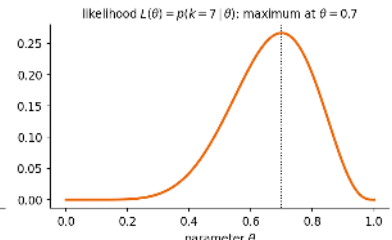
Probability and Likelihood

A model $p(x \mid \theta)$: the probability of data x when the parameters are θ. One formula, read in two ways:

probability: θ fixed, x varies; summed over all outcomes x it gives 1

likelihood $L(\theta) = p(x_{\text{obs}} \mid \theta)$: data fixed, θ varies; not a distribution over θ





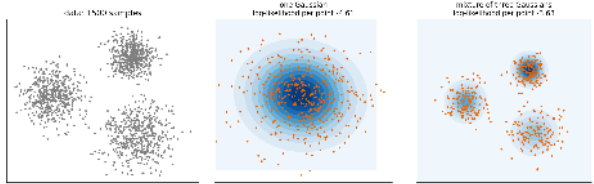
A coin with probability θ of heads, tossed 10 times: $p(k \mid \theta) = \binom{10}{k}\theta^k(1-\theta)^{10-k}$; $p(7 \mid 0.5) = 0.117$. With 7 heads observed, $L(\theta)$ peaks at $\theta = 0.7$, and $\int_0^1 L\, d\theta = 1/11$, not 1.

Maximum Likelihood

Independent samples $\mathbf{x}_1, \ldots, \mathbf{x}_n$; a model $p_\theta(\mathbf{x})$. The likelihood of all the data, and its logarithm:

$$L(\theta) = \prod_{i=1}^{n} p_\theta(\mathbf{x}_i), \qquad \ell(\theta) = \sum_{i=1}^{n} \log p_\theta(\mathbf{x}_i), \qquad \theta^* = \arg\max_\theta \ell(\theta)$$

The logarithm turns the product into a sum and does not move the maximum. One Gaussian: θ^* is the sample mean and variance (with $1/n$).



1500 points from three groups; blue: fitted density, orange: 400 new samples. Mean log-likelihood per point: -4.61 (one Gaussian), -3.63 (mixture of three).

The Kullback–Leibler Divergence

Mismatch of a model q with the true p:

$$\mathrm{KL}(p \,\|\, q) = \mathbb{E}_{x\sim p}\Big[\log \frac{p(x)}{q(x)}\Big]$$

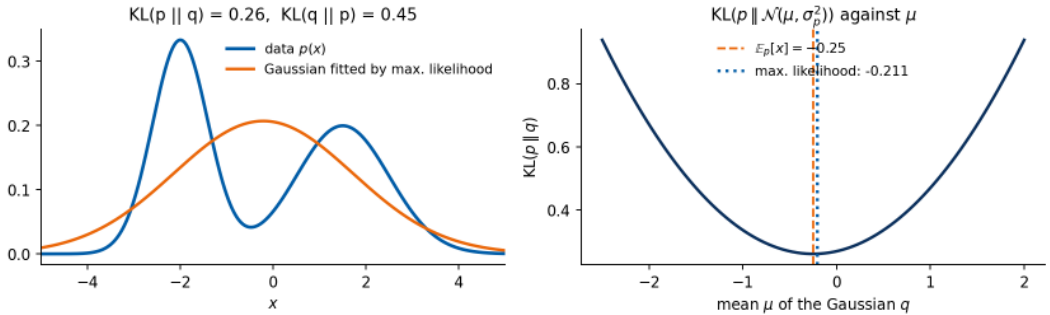
≥ 0, $= 0$ only if $q = p$, not symmetric (so not a distance). With $-\log q(x)$ the surprise of seeing x under q: the average extra surprise when data from p are described by q.

With $q = p_\theta$, split the logarithm:

$$\mathrm{KL}(p \,\|\, p_\theta) = \underbrace{\mathbb{E}_p[\log p]}_{\text{no } \theta} - \underbrace{\mathbb{E}_p[\log p_\theta]}_{\approx \ell(\theta)/n}$$

maximum likelihood = minimum $\mathrm{KL}(p \,\|\, p_\theta)$

Maximum Likelihood and the KL Divergence

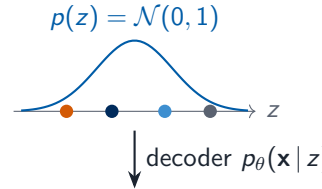


One Gaussian (width fixed at σ_p, the std of p) fitted to the two-peaked p: the KL is smallest at $\mu = \mathbb{E}_p[x] = -0.25$; maximum likelihood on 10^4 samples gives -0.21.

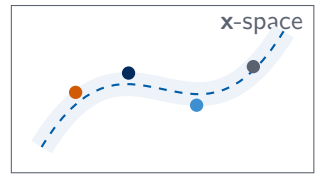
Latent-Variable Models

A simple hidden $\mathbf{z}$ and a map to the data:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$



- **generate**: draw $\mathbf{z} \sim p(\mathbf{z})$, then $\mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z})$
- Gaussian mixture: $z \in \{1, \ldots, K\}$ with $p(z = k) = \pi_k$, $p(\mathbf{x} \mid z = k) = \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$
- VAE: $\mathbf{z} \in \mathbb{R}^d$, $p(\mathbf{z}) = \mathcal{N}(0, I)$, and a neural network (the decoder) sets $p_\theta(\mathbf{x} \mid \mathbf{z})$



each z gives a small cloud around a point of a curve in data space

2

Variational Autoencoders

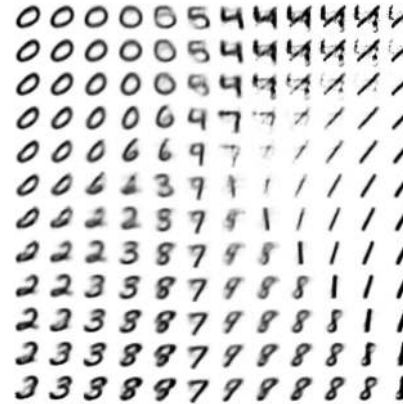
JAGIELLONIAN UNIVERSITY
IN KRAKOW

An Autoencoder Cannot Generate

Can the decoder of an autoencoder produce *new* digits? Pick a latent point $\mathbf{z}$ and compute $G(\mathbf{z})$.

- the autoencoder only learned to rebuild its training images; nothing tells us where in latent space the valid $\mathbf{z}$ are
- between and around the training codes there are regions that decode to images that are no digit

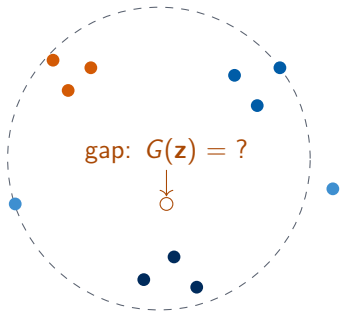
To generate, we need a latent space with a known distribution that we can sample from.



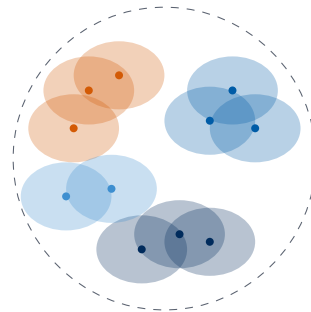
autoencoder with $d = 2$: images decoded from a regular grid of latent points

Latent Points and Latent Clouds

autoencoder: one point per input



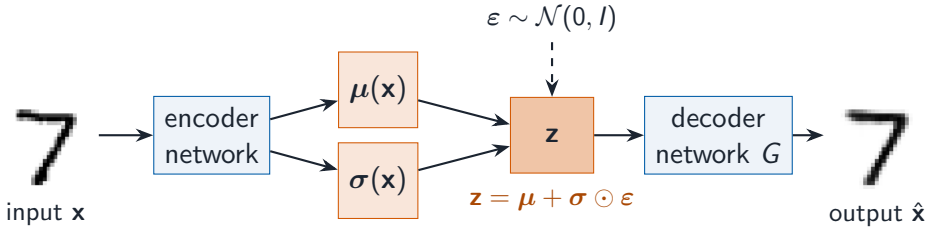
VAE: a Gaussian cloud per input



dashed circle: where $\mathcal{N}(0, I)$ puts most of its probability

Colours: four kinds of input. The VAE encoder gives each input a cloud $\mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \operatorname{diag} \boldsymbol{\sigma}^2(\mathbf{x}))$; the clouds overlap and fill $\mathcal{N}(0, I)$, so a random $\mathbf{z} \sim \mathcal{N}(0, I)$ lands where the decoder was trained.

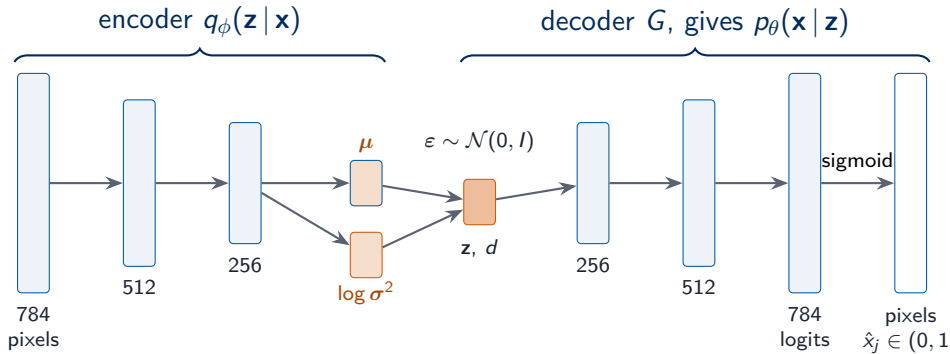
Variational Autoencoder



- an autoencoder maps an input to one *point* $\mathbf{z}$; a VAE maps it to a Gaussian *cloud* centred at $\boldsymbol{\mu}(\mathbf{x})$ with widths $\boldsymbol{\sigma}(\mathbf{x})$ (one μ_k, σ_k for each of the d latent coordinates)
- during training a random point of the cloud is decoded, so every point near $\boldsymbol{\mu}(\mathbf{x})$ must decode to something like $\mathbf{x}$
- to *generate*, skip the encoder: draw $\mathbf{z} \sim \mathcal{N}(0, I)$ and compute $G(\mathbf{z})$

D. P. Kingma, M. Welling, *Auto-Encoding Variational Bayes*, ICLR 2014, arXiv:1312.6114

The VAE on MNIST: Layers



Dense layers with ReLU (bar heights schematic); logits: the outputs before the sigmoid. ϕ: weights of the encoder, θ: of the decoder. The encoder outputs $\log \boldsymbol{\sigma}^2$, which can take any sign; $d = 2$ or 16. Trained 20 epochs with Adam on the 60 000 training digits, without labels.

The Decoder Defines $p_\theta(\mathbf{x} \mid \mathbf{z})$

The decoder ends with a sigmoid, $\hat{\mathbf{x}} = G(\mathbf{z}) \in (0,1)^{784}$ (its last layer outputs *logits*, the numbers before the sigmoid); $\hat{x}_j$ is the probability that pixel j is ink. For black/white pixels $x_j \in \{0,1\}$, drawn independently given $\mathbf{z}$:

$$p_\theta(\mathbf{x} \mid \mathbf{z}) = \prod_{j=1}^{784} \hat{x}_j^{\,x_j} \, (1-\hat{x}_j)^{1-x_j} \quad \Longrightarrow \quad -\log p_\theta(\mathbf{x} \mid \mathbf{z}) = -\sum_{j=1}^{784} \Big[x_j \log \hat{x}_j + (1-x_j) \log(1-\hat{x}_j) \Big]$$

the reconstruction loss (binary cross-entropy) is minus a log-likelihood. MNIST pixels are grey, $x_j \in [0,1]$: the same formula is still used as the loss.

Gaussian Pixels and Independent Pixels

Gaussian pixels give the squared error

$p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x}; G(\mathbf{z}), \sigma_x^2 I)$:

$$-\log p_\theta = \frac{\|\mathbf{x} - G(\mathbf{z})\|^2}{2\sigma_x^2} + \text{const}$$

Independent pixels

given $\mathbf{z}$, the product treats pixels as unrelated: all correlations between pixels must come from $\mathbf{z}$.

The Evidence Lower Bound

Maximum likelihood needs $\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$: an integral over all $\mathbf{z}$, out of reach. The encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ approximates the posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. Since $p_\theta(\mathbf{x}) = p_\theta(\mathbf{x}, \mathbf{z}) / p_\theta(\mathbf{z} \mid \mathbf{x})$ for every $\mathbf{z}$, average log of it over $\mathbf{z} \sim q_\phi$ and multiply inside by q_ϕ / q_ϕ:

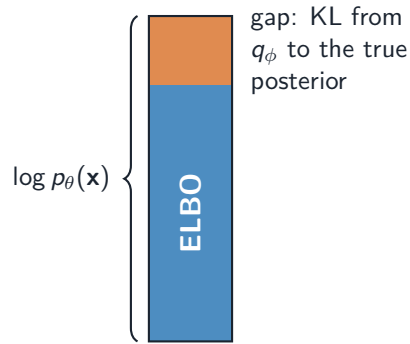
$$\log p_\theta(\mathbf{x}) = \mathbb{E}_{q_\phi}\Big[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}\Big] + \mathbb{E}_{q_\phi}\Big[\log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\Big]$$

With $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$:

$$\log p_\theta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big] - \mathrm{KL}(q_\phi \,\|\, p(\mathbf{z}))}_{\text{ELBO}} + \underbrace{\mathrm{KL}(q_\phi \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x}))}_{\geq 0}$$

Lower Bound and VAE Loss

- KL ≥ 0, so ELBO $\leq \log p_\theta(\mathbf{x})$: a lower bound on the log-likelihood
- VAE loss $= -$ELBO $=$ reconstruction $+$ KL to $\mathcal{N}(0, I)$
- $\mathbb{E}_{q_\phi}$ is estimated with one $\mathbf{z}$ per input
- raising the ELBO raises the likelihood or closes the gap



$d = 16$ VAE, averaged over the training digits: $-$ELBO $= 99.9 = 80.3$ (reconstruction) $+ 19.6$ (KL)

The Reparametrisation Trick

Training needs $\partial\mathcal{L}/\partial\boldsymbol{\mu}$ and $\partial\mathcal{L}/\partial\boldsymbol{\sigma}$, but $\mathbf{z}$ is *drawn at random*: "draw $\mathbf{z}$ from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$" has no derivative with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

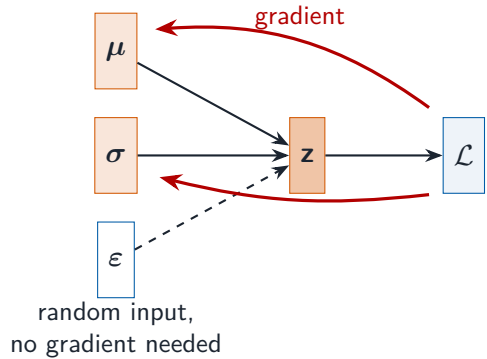
Move the randomness out of the network:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

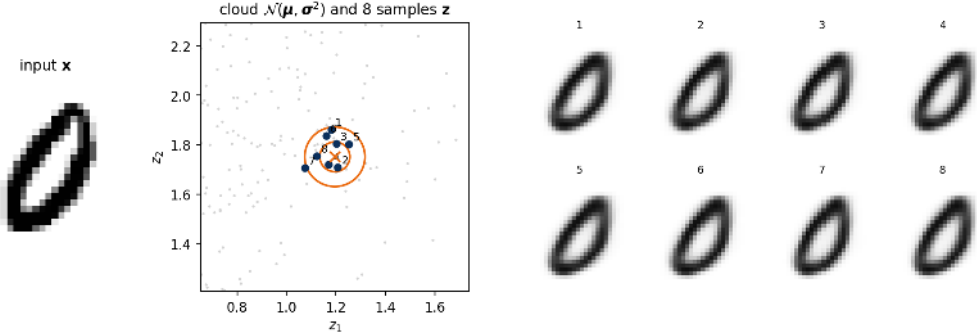
ε is an input that does not depend on the weights; $\mathbf{z}$ is an ordinary function of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$:

$$\frac{\partial z_k}{\partial \mu_k} = 1, \qquad \frac{\partial z_k}{\partial \sigma_k} = \varepsilon_k,$$

and backpropagation runs from the loss through the decoder, $\mathbf{z}$ and the encoder.



Samples from the Cloud of One Input



One test digit (the 0 that the $d = 2$ VAE rebuilds best; with two latent numbers some unusual digits are rebuilt as another digit). The encoder gives its cloud: centre $\boldsymbol{\mu}$ (cross), widths $\boldsymbol{\sigma}$ (ellipses: 1 and 2σ). Eight noises $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ give $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ (dots); the decoder turns each into a version of the input. In training, one such $\mathbf{z}$ per input is decoded and compared with $\mathbf{x}$; autograd gives $\partial z_1/\partial \mu_1 = 1$, $\partial z_1/\partial \sigma_1 = \varepsilon_1$.

The VAE Loss and the Latent Space

Per input $\mathbf{x}$, with one $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon$:

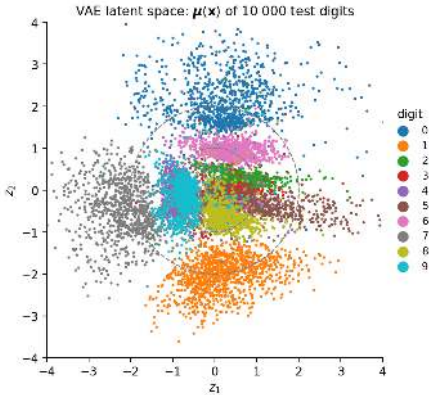
$$\mathcal{L}(\mathbf{x}) = \underbrace{-\log p_\theta(\mathbf{x} \mid \mathbf{z})}_{\text{rebuild the input (BCE)}} + \underbrace{\frac{1}{2} \sum_{k=1}^{d} (\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1)}_{\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\, \boldsymbol{\sigma}^2) \,\|\, \mathcal{N}(0, I))}$$

Each KL term is 0 only at $\mu_k = 0$, $\sigma_k = 1$, and grows when the cloud moves away from the origin or shrinks to a point.

The two terms pull apart: reconstruction wants each cloud small and far from the others; the KL pulls every cloud to $\boldsymbol{\mu} = 0$, $\boldsymbol{\sigma} = 1$. Balance: clouds that overlap and fill $\mathcal{N}(0, I)$, without gaps.

What the latent space learns: a coordinate system for the data; similar inputs get nearby clouds, moving through $\mathbf{z}$ changes the output smoothly, and a $\mathbf{z} \sim \mathcal{N}(0, I)$ lies where some training cloud was.

The Latent Space of a VAE

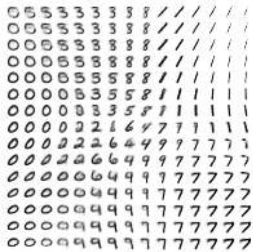


Latent representation: the encoder compresses $\mathbf{x} \in \mathbb{R}^{784}$ into $\mathbf{z} \in \mathbb{R}^d$, coordinates in which similar images are close.

VAE with $d = 2$, trained without labels; each point: the centre $\boldsymbol{\mu}(\mathbf{x})$ of one test digit's cloud, coloured by its label.

- digits of one class gather: the latent space organises the data
- 80% of the centres lie within radius 2 (dashed: radius 1, 2), against 86% for $\mathcal{N}(0, I)$; classes 0, 1, 7 sit at the edge

New Digits from the VAE



$d = 2$: images decoded from a grid of $\mathbf{z}$ over $[-2.5, 2.5]^2$



$d = 16$: 32 new digits, each from a random $\mathbf{z} \sim \mathcal{N}(0, I)$

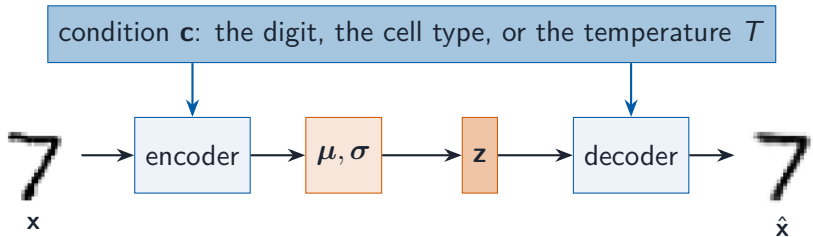
- continuous latent space: neighbouring $\mathbf{z}$ give similar digits
- new, not memorised: on average a generated digit is as far from its nearest training image (Euclidean distance in pixel space, 4.73) as a real test digit the network never saw (4.03); somewhat blurry, a known property of VAEs

3

Conditional Generation

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Conditional VAE



$\mathbf{c}$ is an extra input of *both* networks, $q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{c})$ and $p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})$; the model is $p_\theta(\mathbf{x} \mid \mathbf{c}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})\, p(\mathbf{z})\, d\mathbf{z}$, with the same loss as the VAE. To generate: fix $\mathbf{c}$, draw $\mathbf{z} \sim \mathcal{N}(0, I)$, compute $G(\mathbf{z}, \mathbf{c})$.

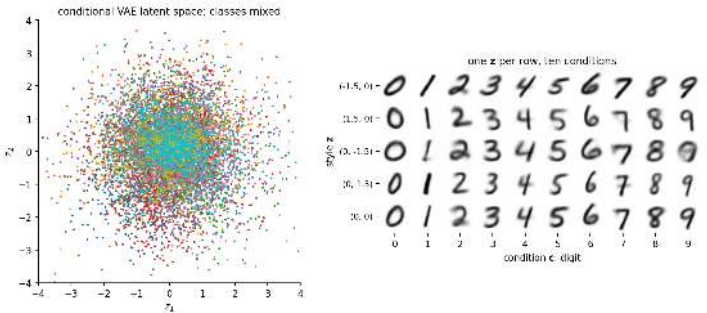
K. Sohn, H. Lee, X. Yan, *Learning Structured Output Representation using Deep Conditional Generative Models*, NeurIPS 2015, NeurIPS proceedings

Condition and Latent Vector

- $\mathbf{c}$ tells the decoder *what* to draw. Every bit stored in $\mathbf{z}$ costs KL, while what $\mathbf{c}$ supplies is free, so $\mathbf{z}$ keeps only what $\mathbf{c}$ does not say: the *style* (slant, thickness, size)

- a class k enters *one-hot*, $\mathbf{e}_k = (0, \ldots, 0, 1, 0, \ldots, 0)$ with the 1 at position k, not as the number k, which would impose an order on the classes; physical parameters such as (T, h) enter as numbers

Latent Space of the Conditional VAE



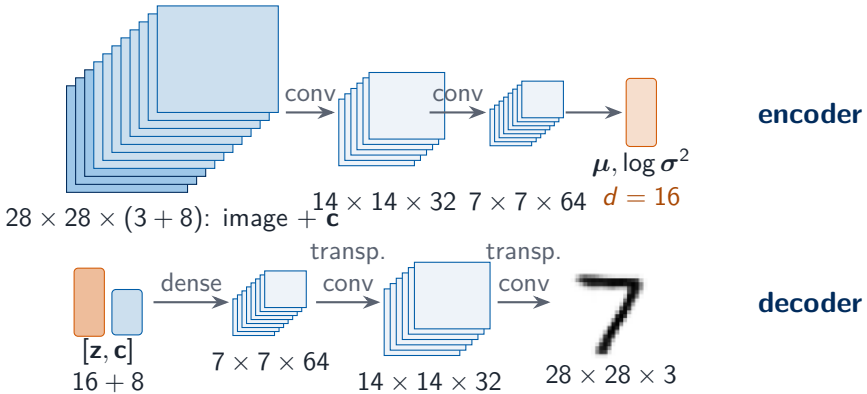
Conditional VAE with $d = 2$, label given as $\mathbf{c}$. Left: the centres $\boldsymbol{\mu}(\mathbf{x}, \mathbf{c})$ of the test digits, coloured by digit, are mixed: the position no longer tells the class. Right: one fixed $\mathbf{z}$ per row, the ten conditions along the row: the digit changes, slant and thickness stay. **c chooses what, z chooses how.**

Conditional VAE on MNIST



- each row: $\mathbf{c}$ = one digit, 0 to 9; each column: one random $\mathbf{z} \sim \mathcal{N}(0, I)$, the *same* in every row
- along a column the style stays and the digit changes: $\mathbf{z}$ carries the style, $\mathbf{c}$ the identity
- the generated digits are checked with a classifier of real digits

The Condition in a Convolutional Network



BloodMNIST: the one-hot cell type enters as 8 extra constant channels (1 for the requested type, 0 otherwise); stride-2 convolutions halve the size, transposed convolutions double it.

Generating a Chosen Digit

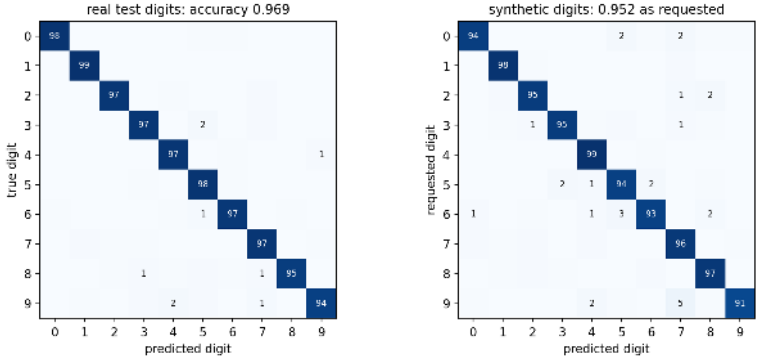
Only the decoder: condition $\mathbf{c} = \mathbf{e}_k$ (one-hot digit k), random style $\mathbf{z} \sim \mathcal{N}(0, I)$, image sigmoid $\big(G(\mathbf{z}, \mathbf{c})\big)$.

```
def generate_digit(k, n=10):
    c = F.one_hot(torch.full((n,), k),
                  10).float()
    z = torch.randn(n, cvae.d)
    with torch.no_grad():
        return torch.sigmoid(
            cvae.decode(z, c))
```

Row k: `generate_digit(k, 12)`; each column a new $\mathbf{z}$.

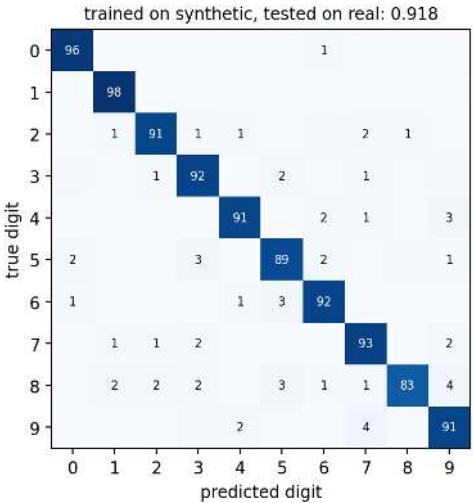


Classifier Check of Synthetic Digits



Classifier $784 \to 256 \to 10$, trained on real digits only. Left: on the real test digits (row: true digit). Right: on 1000 generated digits per class (row: requested digit); entries in % of the row. Diagonal mean: 0.95; the confusions resemble those of real digits.

Training on Synthetic Digits Only



The same classifier, trained on 60 000 *generated* digits (6000 per class, labels = requested digits), tested on the real test digits:

- trained on real digits: accuracy 0.969
- trained on synthetic digits: accuracy 0.918

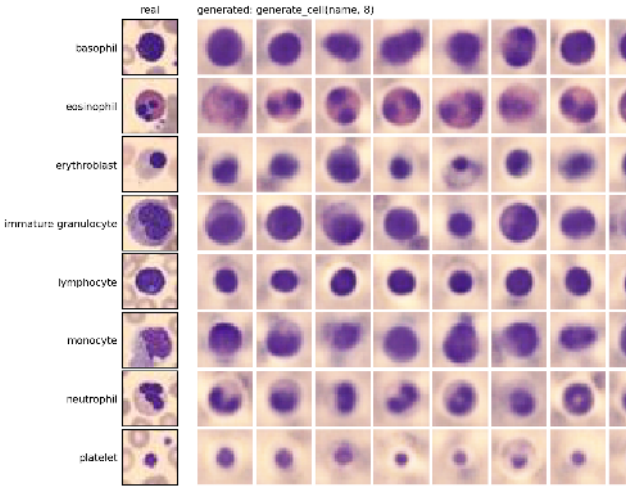
The gap measures what the generator misses of the real distribution: styles it never produces, and blurred strokes.

Generating a Cell of a Chosen Type

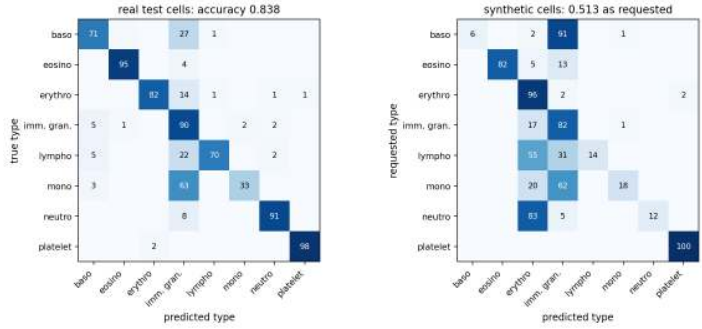
The same recipe, with the cell type as condition and an RGB output:

```
def generate_cell(name, n=8):
    k = names.index(name)
    c = F.one_hot(torch.full((n,),
                  8).float()
    z = torch.randn(n, bcvae.d)
    with torch.no_grad():
        return torch.sigmoid(
            bcvae.decode(z, c))
```

First column: one real cell of the type; then 8 generated.

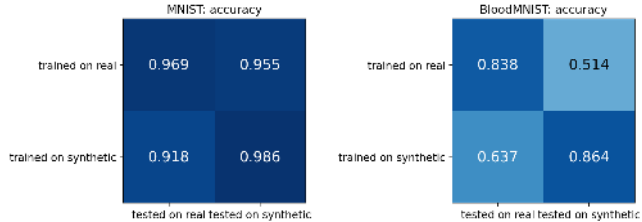


Classifier Check of Synthetic Cells



CNN trained on real cells only. Left: real test cells (accuracy 0.84). Right: 500 generated cells per requested type (diagonal mean 0.51). Platelets (100%) and erythroblasts (96%), told apart by size, come out right; generated basophils are taken for immature granulocytes (91%), neutrophils for erythroblasts (83%): the VAE blurs the shape of the nucleus.

Training and Testing on Real and Synthetic Data



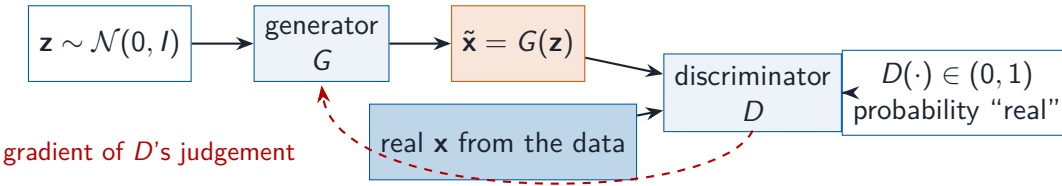
- **trained on real, tested on synthetic:** does the generator draw the requested class?
- **trained on synthetic, tested on real:** do the synthetic data carry what is needed to recognise real instances?
- MNIST: synthetic digits almost replace real ones (0.92 against 0.97); blood cells: trained on synthetic, 0.64 on real cells (0.84 with real training) but 0.86 on synthetic: types easy to tell apart, but not the real ones

4

Generative Adversarial Networks

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Generative Adversarial Network



- **generator** G: turns random noise $\mathbf{z}$ into a sample $\tilde{\mathbf{x}} = G(\mathbf{z})$; it never sees the data
- **discriminator** D: a binary classifier, $D(\mathbf{x})$ = probability that $\mathbf{x}$ comes from the data and not from G
- trained against each other: G learns from D's gradient to produce samples that D takes for real
- no formula for $p_G(\mathbf{x})$: the GAN learns to *sample*, not to evaluate a probability

I. J. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014, arXiv:1406.2661

The GAN Loss

D is an ordinary classifier (real $= 1$, generated $= 0$); G is trained to make it wrong. One value, maximised by D and minimised by G:

$$V(D,G) = \underbrace{\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\big[\log D(\mathbf{x})\big]}_{D \text{ recognises real data}} + \underbrace{\mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,I)}\big[\log\big(1 - D(G(\mathbf{z}))\big)\big]}_{D \text{ recognises generated data}}, \qquad \min_G \max_D V(D,G)$$

$-V$: the binary cross-entropy of D; G now denotes the generator. Training alternates gradient steps of D and of G.

- in practice G minimises $-\log D(G(\mathbf{z}))$, not $\log(1 - D(G(\mathbf{z})))$: same optimum, strong gradient when D rejects the samples easily, early in training (used in our notebook)
- with the best D, $V = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \,\|\, p_G)$, Jensen–Shannon divergence $\mathrm{JS}(p \,\|\, q) = \frac{1}{2}\mathrm{KL}(p \,\|\, m) + \frac{1}{2}\mathrm{KL}(q \,\|\, m)$, $m = \frac{1}{2}(p+q)$: 0 only at $p_G = p_{\text{data}}$, where the game ends

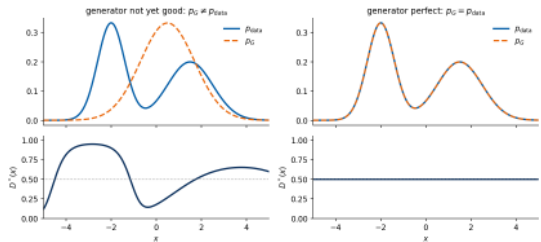
I. J. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014, arXiv:1406.2661

The Best Discriminator

For a fixed generator with density p_G, the D that maximises V is

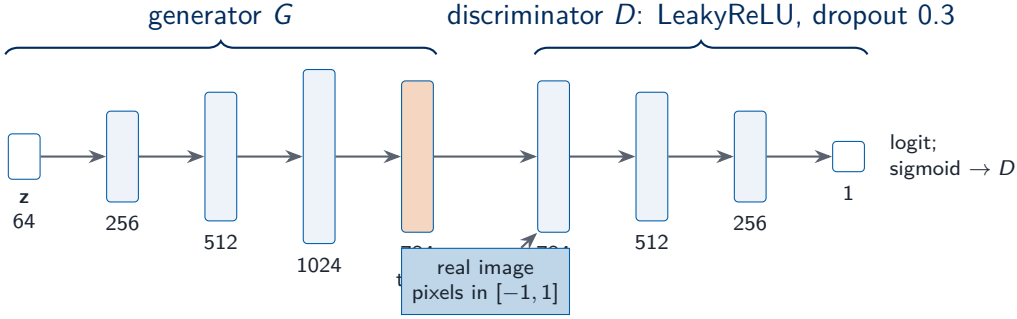
$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}$$

the probability that a point at $\mathbf{x}$ is real: at each $\mathbf{x}$, D maximises $a \log y + b \log(1-y)$, $a = p_{\text{data}}(\mathbf{x})$, $b = p_G(\mathbf{x})$, with maximum at $y = a/(a+b)$. $D^* > \frac{1}{2}$: G puts too little probability there; $D^* < \frac{1}{2}$: too much; $p_G = p_{\text{data}}$: $D^* = \frac{1}{2}$ everywhere.



I. J. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014, arXiv:1406.2661 (Proposition 1)

GAN on MNIST: the Networks

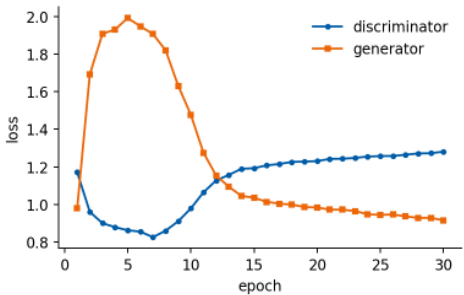


One training step, on a mini-batch of real images and of $G(\mathbf{z})$:

1. update D: binary cross-entropy with labels real $= 1$, generated $= 0$
2. update G, with D frozen: label its samples "real" $(= 1)$ and backpropagate through D into G (Adam, learning rate $2 \cdot 10^{-4}$, 30 epochs)

A GAN on MNIST



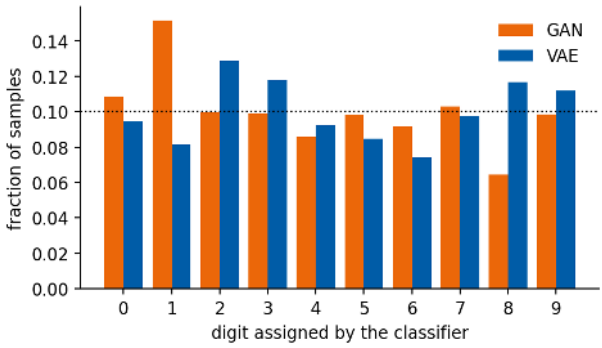


- top: the same 16 random $\mathbf{z}$ after 1, 5 and 30 epochs; the samples sharpen
- the loss curves do not show it: they measure a game between two networks, not the quality of the samples

Difficulties of GAN Training

- **a game, not a minimisation:** the solution is a saddle point (a maximum for D and a minimum for G at the same time); no loss decreases steadily, the two networks can oscillate, and the loss curves say little about the quality of the samples
- **vanishing gradients:** when D separates real from generated perfectly, $\log(1 - D(G(\mathbf{z})))$ saturates and G gets almost no gradient
- **mode collapse:** G finds a few samples that fool D and produces only those, e.g. only some of the digits
- **sensitivity:** learning rates, activations and the balance between G and D matter; our first version (ReLU, sigmoid output, no dropout) collapsed onto a single blob within one epoch

Mode Collapse



- the MNIST classifier labels 2000 new samples of the GAN and 2000 of the VAE
- a generator that covers the data produces each digit about as often as in MNIST, roughly 10% (dotted line; from 9% for 5s to 11% for 1s)
- this GAN produces every digit, between 6% and 15% of the samples: no mode collapse

The Wasserstein Distance

On a line: the area between the cumulative distribution functions F,

$$W(p, q) = \int |F_p(x) - F_q(x)|\, dx,$$

the amount of probability moved times the distance it travels (earth mover's distance).

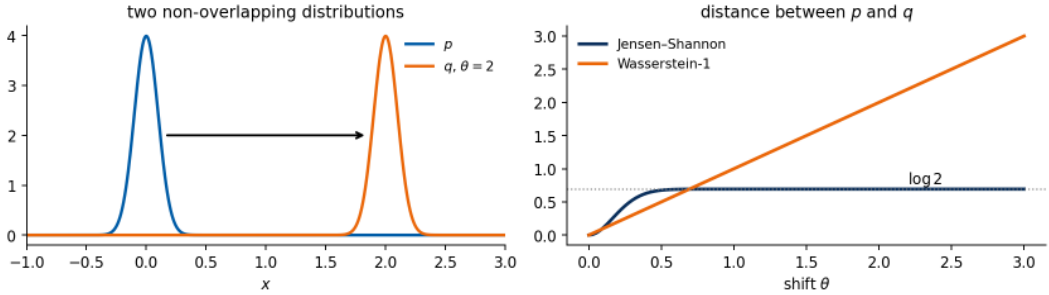
In general (Kantorovich–Rubinstein):

$$W(p, q) = \max_{\|f\|_L \le 1} \mathbb{E}_p[f] - \mathbb{E}_q[f]$$

over 1-Lipschitz f: $|f(x) - f(y)| \le |x - y|$. In a WGAN the **critic** f maximises $\mathbb{E}_{\text{data}}[f] - \mathbb{E}_{\mathbf{z}}[f(G(\mathbf{z}))]$ (a score, not a probability; the slope limit is kept by a gradient penalty), and G minimises $-\mathbb{E}_{\mathbf{z}}[f(G(\mathbf{z}))]$.

M. Arjovsky, S. Chintala, L. Bottou, *Wasserstein GAN*, ICML 2017, arXiv:1701.07875; I. Gulrajani et al., *Improved Training of Wasserstein GANs*, NeurIPS 2017, arXiv:1704.00028

Wasserstein and Jensen–Shannon Divergence

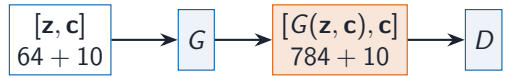


Two bumps a distance θ apart. The Jensen–Shannon divergence (what the original GAN loss measures at the best D) stays at $\log 2$ once they do not overlap: no gradient. $W = \theta$: it says how far, at every distance.

M. Arjovsky, S. Chintala, L. Bottou, *Wasserstein GAN*, ICML 2017, arXiv:1701.07875; I. Gulrajani et al., *Improved Training of Wasserstein GANs*, NeurIPS 2017, arXiv:1704.00028

Conditional GAN





- **c** (one-hot digit) is appended to the input of *both* networks; $D(\mathbf{x}, \mathbf{c})$ judges the *pair*: is this a real image of digit **c**?
- a realistic 3 sent with $\mathbf{c}$ = "7" is rejected, so G must respect the condition
- rows: requested digits 0 to 9; a classifier of real digits recognises 0.96 of 200 samples per digit as requested (conditional VAE, same test: 0.96)

M. Mirza, S. Osindero, *Conditional Generative Adversarial Nets*, arXiv:1411.1784

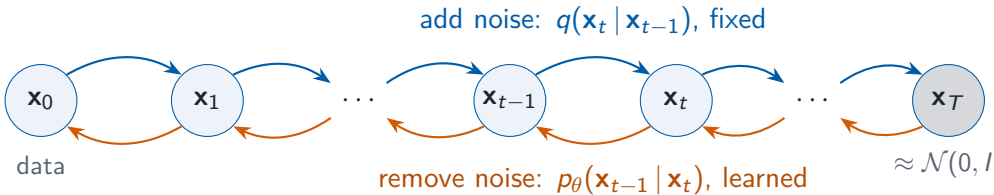
VAE and GAN

	VAE	GAN
objective	maximise the ELBO $\leq \log p_\theta(\mathbf{x})$	$\min_G \max_D V(D, G)$
training	one loss that decreases	a game between two networks
latent space	an encoder maps data to it	no encoder
likelihood	a lower bound on $\log p_\theta(\mathbf{x})$	none
samples	often blurry	sharper edges, but pixel noise and artefacts
failure	blurry averages	unstable training, mode collapse

5
Diffusion Models

JAGIELLONIAN UNIVERSITY
IN KRAKOW

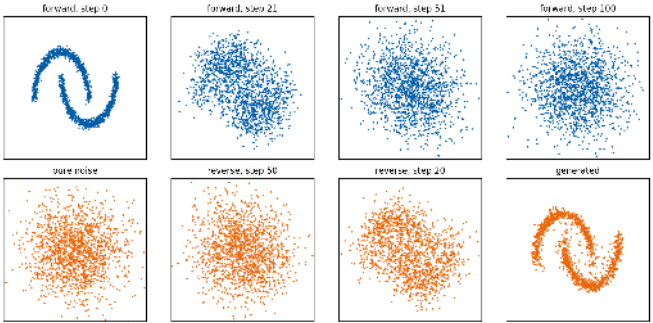
Diffusion Models



Forward: a little Gaussian noise per step until only noise is left (no learning). Reverse: a network removes the noise one step at a time; started from fresh noise, it produces a new sample.

J. Sohl-Dickstein et al., *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*, ICML 2015, arXiv:1503.03585; J. Ho, A. Jain, P. Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS 2020, arXiv:2006.11239

Diffusion on Two Moons



Top row: the forward process destroys the two moons, one small step at a time, until only noise is left. Bottom row: the learned reverse process turns fresh noise into new points on the two moons.

The Forward Process

One step, with a fixed schedule $\beta_1 < \cdots < \beta_T$:

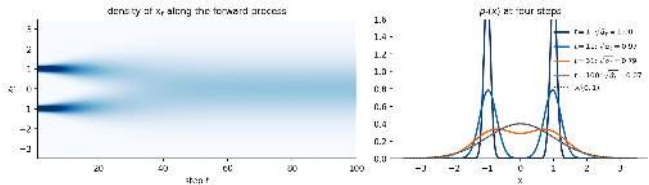
$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I)$$

$(\sqrt{1-\beta_t})^2 + (\sqrt{\beta_t})^2 = 1$: data of variance 1 keep variance 1.

t steps at once (sums of independent Gaussians are Gaussian):

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$$

$\bar{\alpha}_T \approx 0$: the last step is pure noise.

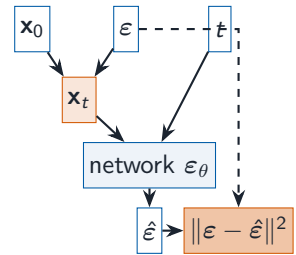


1D data at ± 1, $T = 100$, β_t from 10^{-4} to 0.1: $\sqrt{\bar{\alpha}_T} = 0.075$. The two peaks merge into one $\mathcal{N}(0, 1)$.

Training a Diffusion Model

A data sample $\mathbf{x}_0$, a random step t, a known noise $\varepsilon \sim \mathcal{N}(0, I)$: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$. The network $\varepsilon_\theta(\mathbf{x}_t, t)$ **predicts the noise**:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \varepsilon} \left\| \varepsilon - \varepsilon_\theta(\mathbf{x}_t, t) \right\|^2$$



an ordinary regression: one loss that decreases, stable training.

Generation is slow: many denoising steps. Behind today's image generators, and in science: protein structures (RFdiffusion), crystals (MatterGen), weather ensembles (GenCast).

J. L. Watson et al., *De novo design of protein structure and function with RFdiffusion*, Nature **620**, 1089 (2023), doi:10.1038/s41586-023-06415-8; C. Zeni et al., *A generative model for inorganic materials design*, Nature **639**, 624 (2025), doi:10.1038/s41586-025-08628-5; I. Price et al., *Probabilistic weather forecasting with machine learning*, Nature **637**, 84 (2025), doi:10.1038/s41586-024-08252-9

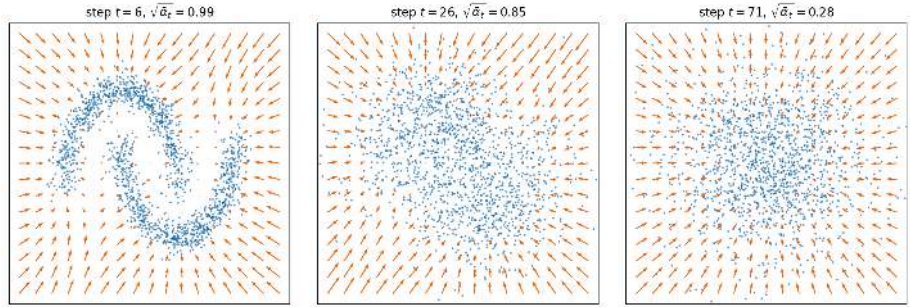
The Reverse Step

From $\mathbf{x}_T \sim \mathcal{N}(0, I)$, for $t = T, \dots, 1$, with $\alpha_t = 1 - \beta_t$ and fresh $\boldsymbol{\xi} \sim \mathcal{N}(0, I)$ (none at $t = 1$):

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon_\theta(\mathbf{x}_t, t)\Big) + \sqrt{\beta_t}\,\boldsymbol{\xi}$$

From $\hat{\varepsilon}$ the clean sample can be estimated, $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\varepsilon})/\sqrt{\bar{\alpha}_t}$, and the step moves $\mathbf{x}_t$ a little towards it, plus fresh noise. The predicted noise is also a direction, with p_t the distribution of $\mathbf{x}_t$: $-\varepsilon_\theta/\sqrt{1-\bar{\alpha}_t} \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, the **score**, which points to higher density of the noisy data.

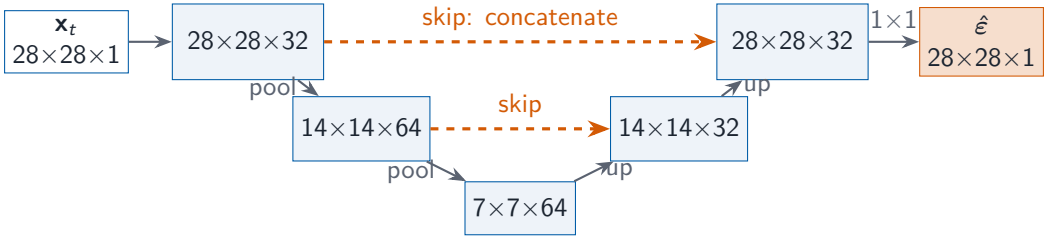
The Learned Score



Blue: noisy two-moons data at three steps; orange: the learned arrows. At every step they point from empty regions towards the noisy data, which change from two moons (small t) to one broad cloud (large t).

Y. Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021, arXiv:2011.13456

The U-Net Noise Predictor

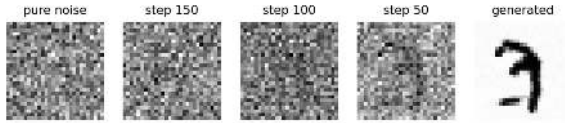


step $t \to$ 64 numbers $\sin(\omega_k t), \cos(\omega_k t)$ at 32 frequencies ω_k:
a smooth code of t, added to the features in every block

Each block: two 3×3 convolutions; the final 1×1 convolution mixes the 32 channels of each pixel into one number. The shrinking path sees larger regions; the skip connections carry the fine details to the growing path. The network predicts the noise of every pixel.

O. Ronneberger, P. Fischer, T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015, arXiv:1505.04597

Diffusion on MNIST



one generated digit, from pure noise through the reverse steps



- $T = 200$ steps, β_t from $5 \cdot 10^{-4}$ to 0.1: $\sqrt{\bar{\alpha}_T} = 0.0055$; one image = 200 passes of the U-Net
- trained on the 60 000 training digits in about 26 minutes on a CPU
- all ten digits appear, most readable (mean classifier confidence 0.80)

6

Generating Ising Configurations

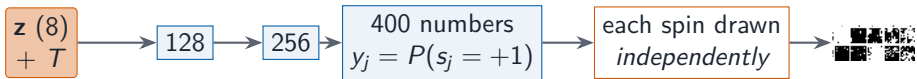


JAGIELLONIAN UNIVERSITY
IN KRAKOW

Generating Spin Configurations at a Chosen Temperature

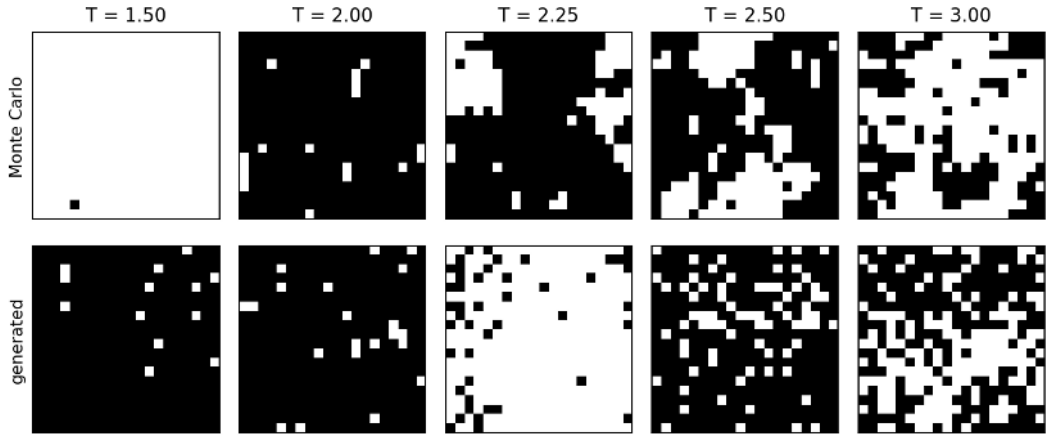
For spins the true distribution is known, $p(\mathbf{s} \mid T) = e^{-H(\mathbf{s})/T}/Z$, $H(\mathbf{s}) = -\sum_{\langle ij \rangle} s_i s_j$ (sum over nearest-neighbour pairs, $J = k_B = 1$; Lecture 4); Metropolis Monte Carlo samples it, exactly in the limit of a long run. Can a network learn it from samples alone?

- **data:** Monte Carlo configurations of a 20×20 lattice, 51 temperatures from 1.0 to 3.5, 200 configurations each
- **model:** conditional VAE, $c = T$, 8 latent numbers; spins ± 1 as pixels $0/1$
- **physical test:** compare $\langle |M| \rangle$ and the energy per spin of the generated configurations with Monte Carlo, at every T



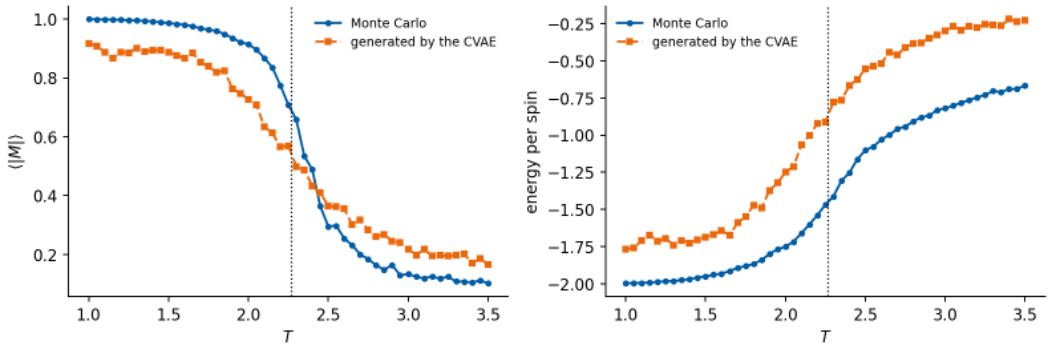
$$p_\theta(\mathbf{s} \mid \mathbf{z}, T) = \prod_{j=1}^{400} y_j^{(1+s_j)/2} (1 - y_j)^{(1-s_j)/2}$$

Generated Spin Configurations



Configurations generated by the conditional VAE at chosen temperatures, next to Monte Carlo configurations at the same temperatures.

Observables of Generated Spin Configurations



$\langle |M| \rangle$ follows the drop around T_c; the energy is too high at every T (largest gap 0.63 per spin, at $T = 2.3$). Given $\mathbf{z}$, the spins are drawn independently, so isolated flipped spins appear where Monte Carlo has compact domains: the nearest-neighbour correlations $\langle s_i s_j \rangle$, which fix the energy, are lost. **A physical test shows what a picture can hide.**

Summary

- a **generative model** learns the distribution $p(\mathbf{x})$ of the data from samples, and draws new samples from it
- **maximum likelihood**: choose the parameters that make the observed data most probable
- **VAE**: a latent space with a known distribution, $\mathcal{N}(0, I)$; decode random $\mathbf{z}$ to generate
- **conditional** models generate what we ask for: a digit, a cell type, a temperature
- **GAN**: a generator against a discriminator; sharper edges than a VAE, hard training; **diffusion**: learn to remove noise in many small steps
- **Ising model**: generated configurations can be checked against the physics, $\langle |M| \rangle(T)$ and the energy

Tutorial: Hands-on

Hands-on

- train the VAE without the KL term and generate from $\mathcal{N}(0, I)$: what goes wrong?
- generate digits with a conditional VAE and walk along one latent coordinate: which property of the digit changes?
- condition the Ising generator on a temperature between two training temperatures: are $\langle |M| \rangle$ and the energy right?

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



